

CONTENTS

- Overview
- 1 Agentic AI
- 2 Large reasoning models
- 3 Vector databases
- 4 Retrieval augmented generation
- 5 Model Context Protocol
- 6 Mixture of experts
- 7 AGI and ASI
- Glossary
- Check yourself
- Where to go next

Seven Terms That Describe a Modern AI System

Place agents, reasoning models, vector search, RAG, MCP, mixture of experts and ASI on one map, and see how they connect.

For engineers who keep meeting these terms and want the mechanism behind each, not the marketing ·
26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- What a large language model does at the input/output level
- A rough idea of what a neural network layer is
- Comfort reading Python

Overview

These seven terms are not a list of unrelated buzzwords. Six of them describe layers of one system, and they stack: a mixture-of-experts architecture makes a large model affordable to serve, reasoning fine-tuning turns that model into one that plans, an agent loop turns the planner into something that acts, MCP gives it a standard way to reach external systems, and retrieval over a vector database gives it facts it was never trained on.

The seventh, artificial superintelligence, is different in kind. It names a hypothesis rather than a component, and it is worth keeping separate from the six precisely because it is not something you can build with today.

What follows takes each term down to the mechanism — what it actually does, what it costs, and where it breaks — because most of these words are used loosely enough that knowing the definition and knowing the thing are quite different states.

KEY TAKEAWAYS

- ✓ An agent is a loop — perceive, reason, act, observe — around a model that is otherwise stateless and reactive.
- ✓ Large reasoning models are fine-tuned to work step by step, trained on problems whose answers can be checked automatically.
- ✓ A vector database stores embeddings so that 'find similar' becomes a distance computation rather than a keyword match.
- ✓ RAG uses that search to inject relevant text into the prompt, giving a model facts it was never trained on.
- ✓ MCP standardises how applications expose tools and data to models, replacing one-off connectors with a common protocol.
- ✓ Mixture of experts activates only a few subnetworks per token, so total parameters and active parameters come apart.
- ✓ MoE saves computation, not memory — every expert still has to be resident even though few are used.
- ✓ AGI and ASI are both theoretical; ASI additionally supposes recursive self-improvement, which is what makes it a different claim.

01 Agentic AI

0:40

Systems that reason and act autonomously toward a goal, cycling through perceive, reason, act and observe instead of answering once.

KEY POINTS

- The cycle is perceive → reason → act → observe, repeated until the goal is met.
- A chatbot answers once; an agent pursues an objective across many steps.
- Nothing agentic lives in the model — the loop, the tools and the history are ordinary code.
- The loop has no inherent stopping condition, so budgets and termination are your responsibility.
- Consequential actions need limits enforced in code, not merely requested in the prompt.

The loop, in the smallest honest form

Three of the four stages are plain software. Only 'reason' is the model. The step budget is not defensive decoration — nothing in the model guarantees it will ever decide it is finished.

```
def run(goal, tools, max_steps=20):
    history = [system_prompt(), user(goal)]

    for _ in range(max_steps):
        reply = model.chat(history, tools=tools) # reason
        if not reply.tool_calls:
            return reply.content # it says it is done

        history.append(reply)
        for call in reply.tool_calls: # act
            try:
                result = tools[call.name](**call.arguments)
            except Exception as e:
                result = f"ERROR: {e}" # failures are observations
            history.append(tool_result(call.id, result)) # observe

    raise BudgetExceeded(goal)
```

02 Large reasoning models

1:23

LLMs given reasoning-focused fine-tuning so they work through a problem step by step before answering.

KEY POINTS

- Reasoning models are LLMs fine-tuned to work step by step before answering.
- Training uses problems with verifiably correct answers — maths, and code a compiler can test.
- Reinforcement learning rewards reaching the correct answer, not producing plausible-sounding reasoning.
- Domains without an automatic checker gain less, because the training loop needs a grader.
- The visible 'thinking' pause is chain-of-thought being generated.
- Reasoning tokens cost latency and money on every call, so depth is a per-task budget decision.

Why verifiable answers make the training loop work

The grader is the whole design. Where one can be written, the model optimises the property you actually care about. Where it cannot, you are back to rewarding text that reads well — which is a different and much weaker signal.

```
def reward(problem, attempt):
    """Programmatic and definitive – this is what makes the loop possible."""
    if problem.kind == "math":
        return float(parse_final(attempt) == problem.answer)
    if problem.kind == "code":
        if not compiles(attempt):
            return 0.0
        return passed(attempt) / problem.total_tests

# works: maths, code, anything with a checkable answer
# does not: "write a good summary" – no definitive right answer
```

03 Vector databases

2:46

Data stored as embeddings rather than raw blobs, so similarity search becomes a mathematical operation.

KEY POINTS

- An embedding model converts content into a vector encoding its meaning.
- Similarity search is a distance computation, not a keyword match.
- Documents sharing no words can still rank as similar.
- The same approach covers text, images and audio.
- Vectors are only comparable within the model that produced them.
- Approximate nearest-neighbour indexes trade a little recall for a lot of speed.

Similarity as geometry

The top match shares no content word with the query. That is the behaviour a keyword index cannot produce, and it is the entire reason vector search exists.

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

query = embed("revenue growth last quarter")

for doc in corpus:
    print(f"{cosine(query, embed(doc)):.3f} {doc}")

# 0.91 "Q4 performance exceeded plan across all regions."
# 0.88 "Quarterly sales rose against the prior period."
# 0.22 "The revenue recognition policy is in appendix B."
#      ^ shares the word "revenue" and is still least relevant
```

04 Retrieval augmented generation

4:09

Use vector search to pull relevant passages and paste them into the prompt, so the model answers from facts it was never trained on.

KEY POINTS

- The retriever embeds the query, searches, and injects the best matches into the prompt.
- The model's weights are never modified — RAG is entirely input-side.
- New knowledge is live as soon as a document is indexed.
- The model can only answer from passages it was given.
- Silent failure: retrieval misses, the model answers from the wrong text, nothing errors.
- Detect it with a gold set of questions whose correct source you already know.

The whole pipeline

Four steps, and the instruction to admit ignorance is what converts a confident guess into an honest refusal when retrieval comes back empty-handed.

```
def answer(question, store, model, k=5):
    hits = store.search(embed(question), k=k)          # 1. retrieve

    context = "\n\n".join(f"[{i+1}] {h.text}"          # 2. augment
                           for i, h in enumerate(hits))

    prompt = (
        "Answer using only the context below. If it does not "
        "contain the answer, say you do not know.\n\n"
        f"<context>\n{context}\n</context>\n\n{question}"
    )

    return model.generate(prompt)                      # 3. generate
```

05 Model Context Protocol

5:33

A standard way for applications to expose tools and data to models, replacing bespoke connectors with one protocol.

KEY POINTS

- MCP standardises how applications expose context, tools and data to models.
- It replaces one-off connectors per model-tool pairing with a single protocol.
- An MCP server fronts an external system; the model's host application is the client.
- Servers expose tools (actions), resources (readable data) and prompts (templates).
- Something to read is a resource; something to do is a tool — conflating them is a common design error.
- The server boundary is where scope and permission should be enforced.

Why a protocol rather than connectors

The arithmetic is the argument. Bespoke integration work grows with the product of clients and systems; a protocol makes it grow with the sum.

without a standard

client A → database
client A → git repo
client A → email
client B → database
client B → git repo
...

$N \times M$ integrations

with MCP

client A ┌
client B ┤ → MCP → database
client C └ → git repo
 → email

write each server once;

any client can use it

$N + M$

06 Mixture of experts

6:15

Split the model into specialised subnetworks and activate only the few each token needs, so size and cost come apart.

KEY POINTS

- MoE splits the model into specialised expert subnetworks.
- A router activates only the experts a given token needs, then merges their outputs.
- Model size scales without a proportional increase in compute.
- Routing is per token, not per task — one sentence uses many experts.
- Total parameters and active parameters are different numbers.
- Memory is sized by total parameters; compute by active ones. MoE saves FLOPs, not VRAM.

Routing and merging, per token

The gate is a small learned layer producing one score per expert. Only the top few run, and their outputs are summed weighted by those scores — which is what makes the merge differentiable and therefore trainable end to end.

```
def moe_layer(x, experts, gate, k=2):
    scores = softmax(gate(x))          # one score per expert
    top = topk_indices(scores, k)      # only these actually run

    out = 0
    for i in top:
        out = out + scores[i] * experts[i](x)  # weighted merge
    return out

# 128 experts defined, 2 evaluated per token.
# the other 126 still occupy memory.
```

07 AGI and ASI

8:17

Two theoretical thresholds. AGI matches human experts across cognitive tasks; ASI goes beyond, with recursive self-improvement.

KEY POINTS

- ASI is theoretical, does not exist, and may never.
- AGI is also theoretical: all cognitive tasks, as well as any human expert.
- Both quantifiers matter — all tasks, and expert-level rather than average.
- ASI additionally supposes recursive self-improvement, which is a different kind of claim.
- Benchmarks measure narrow capabilities; generality is not among them.
- When the terms appear in marketing, ask what was measured, on what, by whom.

The three levels, and what separates them

Only the first row describes anything that exists. The distinction between the second and third is not scale but self-modification, which is why they are separate terms rather than points on one axis.

narrow AI	strong at specific tasks; everything shipping today
	↓
AGI	all cognitive tasks, at the level of a human expert (theoretical)
	↓
ASI	beyond human level, plus recursive self-improvement (theoretical)

the last step is not "more" – it is a system that can rewrite itself

Glossary

Agentic AI

A system that pursues a goal autonomously by cycling through perceiving, reasoning, acting and observing, rather than answering a single prompt.

Large reasoning model

An LLM fine-tuned to work through problems step by step before answering, trained on tasks whose answers can be checked automatically.

Chain of thought

The intermediate reasoning a model generates before its final answer. What a chatbot is producing while it shows that it is thinking.

Embedding model

A model that converts text, images or audio into a vector whose position encodes the content's meaning.

Vector database

Storage built to hold embeddings and retrieve them by similarity, turning 'find things like this' into a distance computation.

RAG

Retrieval augmented generation: searching a corpus for relevant passages and inserting them into the prompt so the model can answer from them.

Silent failure

When retrieval misses the passage containing the answer. No error is raised — the model simply answers from the wrong material, fluently.

MCP

Model Context Protocol: a standard for applications to expose tools, data and prompt templates to models, replacing per-pairing connectors.

MCP server

The component fronting an external system, declaring what a model can reach there and how. The right place to enforce permission.

Tool vs resource

In MCP, a tool is an action the model invokes; a resource is data the client reads. Something to do versus something to read.

Mixture of experts (MoE)

An architecture splitting a model into specialised subnetworks, with a router activating only a few per token.

Router / gating network

The small learned layer that scores experts for each token and selects the top few to run.

Active parameters

The fraction of a MoE model's parameters used for a given token. Determines compute cost, while total parameters determine memory.

AGI

Artificial general intelligence: a theoretical system able to complete all cognitive tasks as well as any human expert.

ASI

Artificial superintelligence: theoretical capability beyond human level, potentially able to redesign and improve itself recursively.

Check yourself

Answer first, then open each one.

Two products use the same model. One is a chatbot and one is an agent. What actually differs?

The control flow around the call. The agent puts the model in a loop with tools and feeds each result back into context so the next turn can use it. Nothing about the model changed, which is why 'is this agentic?' is a question about your code rather than about which model you bought.

Why did reasoning models improve fastest on mathematics and code specifically?

Because their reinforcement learning needs a reward signal, and those domains have verifiable answers — a result that is right or wrong, code that compiles and passes tests. Where you cannot write a programmatic grader you cannot run the loop, so progress there is less direct.

A RAG system returns a fluent, confident, wrong answer, and the logs show no errors. What has probably happened, and why won't monitoring catch it?

Silent failure: retrieval returned plausible but incorrect passages, and the model answered from them. There is nothing to alert on because the request succeeded and the response is well formed. Detecting it requires a gold set of questions with known source passages and a recall measurement you run deliberately.

Why does MCP change integration effort from a product into a sum?

Without a standard, each client-system pairing needs its own connector, so work grows as clients times systems. With MCP each system needs one server and each client speaks the protocol once, so work grows as clients plus systems — and a new client gets every existing server for free.

An MoE model has 100B total and 10B active parameters. What does each number tell you, and which one is easy to misread?

Active parameters determine compute per token, so the model runs roughly as cheaply as a 10B dense model. Total parameters determine memory, because every expert must be resident — you cannot predict which tokens route where. The easy misreading is 'only uses a fraction of its parameters', which sounds like a hosting saving and is not: MoE saves FLOPs, not VRAM.

Why is ASI treated as a different kind of claim from AGI rather than simply more of it?

Because of recursive self-improvement. AGI names a capability level — all cognitive tasks at expert standard. ASI supposes a system that can redesign and upgrade itself, so each version is better at producing the next. That is a qualitative difference in the shape of the claim, not a larger point on the same scale.

Where to go next

- Write an MCP server for one internal system your team uses, and notice how much of the work is deciding what should be a tool versus a resource rather than writing the integration.
- Read the original mixture-of-experts paper (Jacobs, Jordan, Nowlan and Hinton, 1991) and then a modern sparse-MoE transformer paper, and note that the routing idea barely changed while the scale did.
- Instrument expert utilisation on an open MoE model and look for collapse — a few experts taking most of the tokens is the failure the balancing loss exists to prevent.
- Take a task you currently send to a reasoning model and measure whether the thinking tokens change the answer; for classification and extraction they frequently do not, and the latency is pure cost.
- Build the gold-set recall measurement for a RAG system you already run, before touching any prompt — that number is the ceiling on everything above it.
- Trace one agent run end to end and mark which steps were the model and which were ordinary code; the ratio is usually a surprise and tells you where bugs will actually live.
- The vector-database, RAG and agent entries here each have a full lesson elsewhere in this collection — use this page as the map and those for the depth.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.